

EPICODE

Cyber Security & Ethical Hacking

Laboratorio – Cisco CyberOps

Lavorare con File di Testo nella CLI

+ Bonus: Prendere Familiarità con la Shell Linux

Relazione di laboratorio

Studente: Alessandro

Corso: CyberOps / EthicalHacking–Epicode

Giugno 2026

Introduzione

In questa relazione documento lo svolgimento del laboratorio "Lavorare con File di Testo nella CLI" e dell'esercizio bonus "Prendere Familiarità con la Shell Linux", entrambi svolti sulla VM Cisco CyberOps Workstation con l'account utente **analyst**. L'obiettivo principale era prendere confidenza con gli editor di testo grafici e da riga di comando (SciTE e nano), comprendere la struttura e la posizione dei file di configurazione in Linux, e consolidare i comandi di base della shell per la gestione di file e directory (navigazione, copia, spostamento, eliminazione, redirectione dell'output).

Esercizio 1 – Lavorare con File di Testo nella CLI

Parte 1 – Editor di Testo Grafici

Ho effettuato l'accesso alla VM CyberOps come utente analyst e ho avviato SciTE da Applications > CyberOPS > SciTE. Ho digitato la citazione di Douglas Adams nell'area di lavoro e ho salvato il file come **space.txt** nella directory home predefinita. Dopo aver chiuso e riaperto SciTE, ho provato a individuare nuovamente il file tramite File > Open.

Domanda: Sei riuscito a trovare subito space.txt?

Risposta: No. SciTE filtra i file mostrati in base a estensioni conosciute, e .txt non è tra quelle predefinite nell'elenco. Per visualizzare il file ho dovuto cambiare il filtro del tipo di file selezionando "All Files (*)" dal menu a discesa della finestra di apertura.

Parte 2 – Editor di Testo da Riga di Comando

Ho aperto SciTE anche dal terminale con il comando `scite space.txt`, verificando che il file venisse caricato automaticamente al suo interno.

Domanda: Perché il prompt non viene mostrato nel terminale?

Risposta: Perché SciTE è stato avviato come processo in foreground rispetto a quella shell: il terminale resta in attesa che il processo termini prima di restituire il controllo e ripresentare il prompt dei comandi. Finché SciTE è aperto e collegato a quella shell, il prompt non riappare (a meno di interrompere il processo con CTRL+C o avviarlo in background).

Successivamente ho aperto il file con nano (`nano space.txt`), notando che la riga di testo, non contenendo ritorni a capo, viene mostrata come un'unica riga molto lunga che eccede la larghezza dello schermo.

Domanda: Quale carattere usa nano per rappresentare che una linea continua oltre i bordi dello schermo?

Risposta: Il simbolo del dollaro (\$), mostrato all'estremità destra della riga, indica che il testo continua oltre il margine visibile dello schermo.

Parte 3 – Lavorare con i File di Configurazione

Ho esplorato la directory home dell'utente analyst con `ls -l` e poi con `ls -la`, osservando che i file di configurazione utente (come `.bashrc`) sono nascosti per convenzione perché il loro nome inizia con un punto. Ho poi confrontato questa posizione con la directory `/etc`, dove risiedono i file di configurazione dei servizi a livello di sistema.

Domanda: Perché i file di configurazione delle applicazioni utente sono salvati nella directory home dell'utente e non sotto `/etc` con tutti gli altri file di configurazione a livello di sistema?

Risposta: Perché un utente regolare possiede sempre i permessi di scrittura sulla propria directory home, ma non sulla directory `/etc`, che è scrivibile solo dall'utente root. Collocare le configurazioni personali in home permette a ciascun utente di personalizzare il comportamento delle proprie applicazioni senza bisogno di privilegi elevati, mantenendo inoltre separate e indipendenti le impostazioni di utenti diversi sulla stessa macchina.

Ho poi modificato il file `.bashrc` con SciTE cambiando il codice colore del prompt da 32 (verde) a 31 (rosso), salvando il file e aprendo una nuova finestra di terminale per verificare il cambiamento.

Domanda: Anche la finestra del terminale che era già aperta ha cambiato colore da verde a rosso? Spiega.

Risposta: No. Il file `.bashrc` viene letto ed eseguito solo all'avvio di una nuova sessione di shell (apertura di un nuovo terminale o nuovo login). Una shell già aperta ha già caricato in memoria la configurazione precedente e non rilegge automaticamente il file modificato; continua quindi a mostrare il prompt verde finché non viene aperta una nuova shell oppure non si ricarica manualmente la configurazione (ad esempio con il comando `bash` o `source .bashrc`).

Ho ripetuto la stessa modifica da riga di comando con `nano`, cambiando il codice colore da 31 a 33 (giallo) e ricaricando la shell con il comando `bash`, verificando il cambio di colore del prompt.

Parte 3 – Modificare i File di Configurazione per i Servizi (nginx)

Ho aperto il file di configurazione di nginx con `sudo nano -l /etc/nginx/custom_server.conf`, usando `sudo` poiché il file risiede sotto `/etc` ed è modificabile solo da root. Ho cambiato la porta di ascolto da 81 a 8080 (riga 39) e il percorso radice dei contenuti web da `/usr/share/nginx/html/` a `/usr/share/nginx/html/text_ed_lab/` (riga 47), salvando con `CTRL+X`. Ho quindi avviato il servizio con `sudo nginx -c /etc/nginx/custom_server.conf` e verificato la homepage da Firefox all'indirizzo `127.0.0.1:8080`.

Domanda: A cosa si riferisce il messaggio di connessione mostrato nella finestra del terminale dopo l'apertura della homepage?

Risposta: Nel mio caso il messaggio non rappresenta un errore bloccante: nginx, all'avvio con un file di configurazione personalizzato che non specifica percorsi propri per i log, scrive sul terminale un avviso relativo all'apertura dei file di log (`error_log/access_log`), poiché questi non sono stati ridefiniti in `custom_server.conf` e puntano ai percorsi predefiniti di compilazione. Il servizio resta comunque attivo e funzionante, come confermato dal corretto caricamento della homepage nel browser; il messaggio è quindi più un avviso informativo (di tipo `[warn]`) che un vero errore.

Ho poi arrestato il servizio con `sudo pkill nginx` e ho verificato lo stato del server pulendo la cronologia del browser e ricaricando la pagina.

Domanda: Dopo aver arrestato nginx, appare ancora la pagina web all'indirizzo 127.0.0.1:8080?

Risposta: No. Una volta terminato il processo con `pkill`, nginx non è più in ascolto sulla porta 8080: il browser restituisce un errore di connessione ("impossibile connettersi al server"), confermando che il servizio web è stato arrestato correttamente.

Domanda: Domanda Sfida: è possibile modificare il file `/etc/nginx/custom_server.conf` con SciTE? Descrivi il processo.

Risposta: Sì, è possibile, ma SciTE deve essere eseguito con privilegi elevati perché il file si trova sotto `/etc`, dove un utente regolare non ha permessi di scrittura. Il processo consiste nell'aprire un terminale e avviare l'editor con `sudo scite /etc/nginx/custom_server.conf`, inserendo la password di `analyst` quando richiesta. In questo modo il processo SciTE eredita i privilegi di root e può salvare correttamente le modifiche con `File > Save`. Se SciTE venisse aperto senza `sudo`, il file potrebbe comunque essere visualizzato in lettura, ma al momento del salvataggio verrebbe restituito un errore di permesso negato.

Esercizio Bonus – Prendere Familiarità con la Shell Linux

Parte 1 – Basi della Shell

Ho effettuato l'accesso come analyst e ho aperto il terminale dal Dock. Ho consultato le pagine man per orientarmi nella documentazione integrata di Linux.

Domanda: Elenca alcune sezioni incluse in una pagina man.

Risposta: NAME, SYNOPSIS, DESCRIPTION, OPTIONS, EXAMPLES, SEE ALSO e AUTHOR (in alcune pagine man compaiono anche FILES, EXIT STATUS o BUGS).

Domanda: Qual è la funzione del comando cp?

Risposta: Il comando cp copia file o directory da una posizione sorgente a una destinazione, creando una nuova copia e lasciando intatto l'originale.

Domanda: Quale comando useresti per trovare maggiori informazioni sul comando pwd? Qual è la funzione del comando pwd?

Risposta: Userei man pwd. Il comando pwd (print working directory) mostra il percorso completo della directory di lavoro corrente.

Parte 1 – Creare e Cambiare Directory

Domanda: Qual è la directory corrente dopo aver digitato pwd?

Risposta: /home/analyst, ovvero la directory home dell'utente analyst.

Ho creato tre cartelle (cyops_folder1, cyops_folder2, cyops_folder3) con mkdir e mi sono spostato dentro cyops_folder3 con cd.

Domanda: In quale cartella ti trovi dopo il comando cd /home/analyst/cyops_folder3?

Risposta: Nella cartella /home/analyst/cyops_folder3, come confermato anche dal cambiamento del prompt che mostra [analyst@secOps cyops_folder3].

Domanda: Sfida: digita il comando cd ~ e descrivi cosa succede. Perché?

Risposta: Il comando riporta immediatamente alla directory home dell'utente (/home/analyst), indipendentemente dalla posizione di partenza. Questo accade perché il simbolo ~ è un'abbreviazione speciale della shell che fa sempre riferimento alla home directory dell'utente attualmente connesso.

Ho poi creato una sottocartella cyops_folder4 dentro cyops_folder3 ed esplorato il contenuto con ls -la, osservando le voci speciali . e ..

Domanda: Cosa succede digitando `cd .` dalla cartella `cyops_folder3`?

Risposta: Non succede nulla di visibile: la directory corrente resta la stessa, poiché il punto (.) rappresenta la directory corrente stessa, non un livello diverso del filesystem.

Domanda: Cosa succede digitando `cd ..` dalla cartella `cyops_folder3`?

Risposta: La directory corrente cambia, risalendo di un livello alla directory genitore: da `/home/analyst/cyops_folder3` si torna a `/home/analyst`.

Domanda: Quale sarebbe la directory corrente eseguendo `cd ..` da `[analyst@secOps ~]$`?

Risposta: `/home`, cioè la directory genitore di `/home/analyst`.

Domanda: Quale sarebbe la directory corrente eseguendo `cd ..` da `[analyst@secOps home]$`?

Risposta: `/` (la directory radice del filesystem).

Domanda: Quale sarebbe la directory corrente eseguendo `cd ..` da `[analyst@secOps /]$`?

Risposta: Resterebbe `/` (la directory radice). La radice non ha una directory genitore superiore, quindi `cd ..` eseguito dalla radice non produce alcun cambiamento.

Parte 1 – Redirigere e Accodare l'Output

Ho usato `echo` per scrivere un messaggio a schermo e poi ho ripetuto il comando redirigendo l'output verso un file con l'operatore `>`.

Domanda: Dopo aver eseguito `echo ... > some_text_file.txt` non viene mostrato alcun output a schermo. È previsto? Spiega.

Risposta: Sì, è previsto. L'operatore `>` redirige l'intero output del comando dal terminale al file specificato: il testo non viene quindi più mostrato sullo schermo, ma scritto direttamente all'interno di `some_text_file.txt`.

Domanda: Dopo aver rieseguito `echo` con un messaggio diverso e lo stesso operatore `>`, cosa è successo al file di testo? Spiega.

Risposta: Il contenuto precedente del file è stato completamente sovrascritto dal nuovo messaggio. L'operatore `>` tronca il file esistente (o lo crea se non esiste) e scrive il nuovo output, eliminando ciò che era stato salvato in precedenza.

Domanda: Usando invece l'operatore >> per accodare una riga al file, cosa è successo al file di testo? Spiega.

Risposta: La nuova riga è stata aggiunta in coda al contenuto già presente, senza eliminare nulla. L'operatore >> accoda l'output mantenendo intatto il contenuto precedente, diversamente da > che lo sovrascrive.

Parte 1 – Lavorare con i File Nascosti

Domanda: Quanti file vengono visualizzati con ls -l nella directory home?

Risposta: Vengono mostrati solo i file e le directory "visibili", cioè quelli il cui nome non inizia con un punto (nel mio caso le cartelle cyops_folder2, cyops_folder3, Desktop, Downloads, lab.support.files, second_drive e il file space.txt; il numero esatto dipende dallo stato della propria home al momento dell'esecuzione).

Domanda: Quanti file in più vengono visualizzati con ls -la rispetto a ls -l? Spiega.

Risposta: Con ls -la vengono mostrati anche tutti i file nascosti, cioè quelli con nome che inizia con un punto (come .bashrc, .bash_history, .cache, .config), oltre alle due voci speciali . e .. che rappresentano rispettivamente la directory corrente e quella genitore.

Domanda: È possibile nascondere intere directory aggiungendo un punto prima del loro nome? Ci sono directory nascoste nell'output di ls -la?

Risposta: Sì, lo stesso meccanismo dei file si applica anche alle directory: basta far iniziare il nome con un punto per nascondere la directory dalla visualizzazione predefinita. Nell'output di ls -la sono presenti directory nascoste, ad esempio .cache, .config, .gnupg, .local e .mozilla.

Domanda: Fornisci tre esempi di file nascosti mostrati nell'output di ls -la.

Risposta: .bash_history, .bashrc e .bash_logout (altri esempi validi sono .dmrc, .lesshst e .ICEauthority).

Parte 2 – Copiare, Eliminare e Spostare File

Ho copiato some_text_file.txt dalla home a cyops_folder2 con cp, verificando con ls che il file fosse presente in entrambe le posizioni (la copia non elimina l'originale).

Domanda: Identifica i parametri nel comando cp some_text_file.txt cyops_folder2/

Risposta: Il comando ha due parametri: some_text_file.txt è il file sorgente e cyops_folder2/ è la directory di destinazione.

Domanda: Quali sono i file sorgente e destinazione, usando percorsi completi?

Risposta: Sorgente: /home/analyst/some_text_file.txt. Destinazione: /home/analyst/cyops_folder2/some_text_file.txt.

Ho poi rimosso il file dalla home con rm e ho eliminato l'intera cartella cyops_folder1 (con il suo contenuto) usando rm -r, opzione necessaria perché rm da solo non elimina directory.

Infine ho spostato some_text_file.txt da cyops_folder2 nuovamente nella home con mv.

Domanda: Quale comando hai usato per spostare some_text_file.txt da cyops_folder2 alla home?

Risposta: mv cyops_folder2/some_text_file.txt .

Riflessione finale

Domanda: Quali sono i vantaggi dell'utilizzo della riga di comando Linux?

Risposta: La CLI permette di amministrare un sistema anche in assenza di interfaccia grafica, condizione fondamentale per i server remoti "headless" raggiunti via SSH. È inoltre più rapida per attività ripetitive, poiché i comandi possono essere concatenati e automatizzati tramite script; richiede meno risorse di sistema rispetto a un ambiente grafico e offre un controllo più granulare e preciso sulle operazioni svolte. Infine, la sequenza di comandi eseguiti è facilmente documentabile, ripetibile e condivisibile, caratteristiche molto utili in ambito CyberOps per la riproducibilità delle procedure.